How designs are communicated in a team enviroment ?

→ when projects get large scale it is imp for proper comm..
using a common lang.
↓ invent
UML (unified modelling lang)
↓
used to represent the System being built.

UML → Structure diagram (what is system made of) | static
→ Behavior diagram (how it works) | dynamic

System completity → System becomes complex if
(structure is complex e.g
many connections, and behavior
has many interactions)

usecase: A description of how a user
uses the system to achieve a specific goal.

e.g User goal → Borrow a Book
usecase → this whole process is now a usecase.

Documenting usecase is imp ?

→ it prevents confusion
→ each usecase represents a imp feature
→ Ensures nothing is forgotte

Iterative development model → Building a software in small parts instead all at once.

1 part Build → test it → improve

each step/part is Build on whats written in usecase

Not usecase doesnt include in which Lang the program will be written in.

How to find usecases:

1) chose System Boundary → what exactly we are Building / parts inside the System vs outside

online libary System
outside parts → Pc.

2) find primary actors and their goals.
↓
People / System interacting with our System.

Each user goal → 1 usecase

Syntax (Usecase)



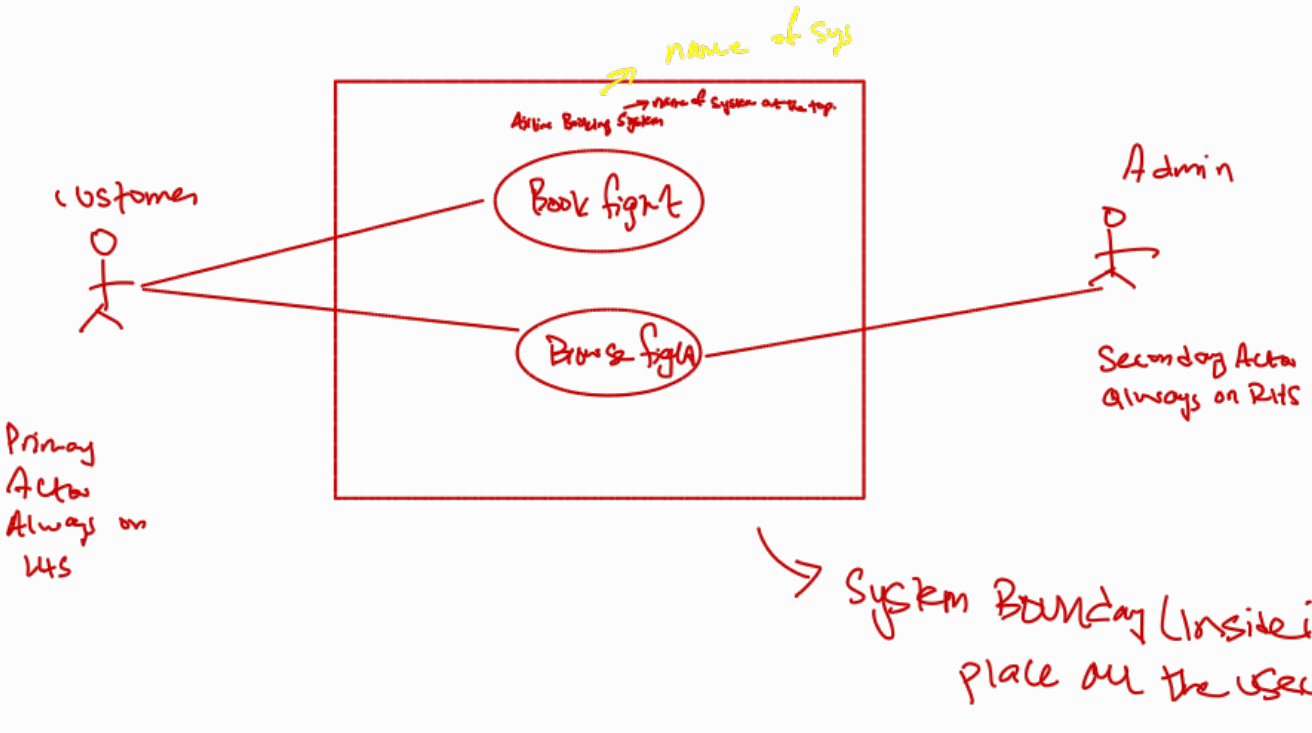
Actors
outside the System Boundary



usecase
↓
naming according to Question

Name of System

Actors connects to ovals



↳ System Boundary (Inside it place all the usecases)

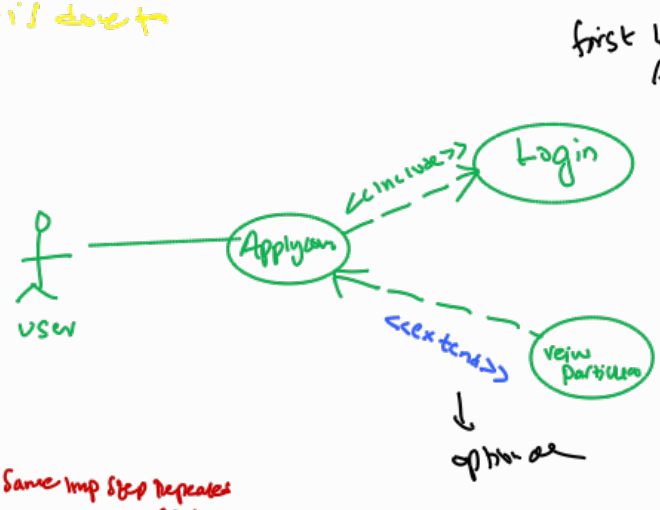
Usecase → what the user wants to accomplish using the System

and not upon coding

Verb-noun phrase

↓ ↘

Action being done — the thing the action is done to

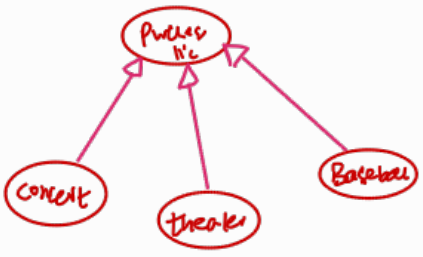


include → same imp step repeated again and..

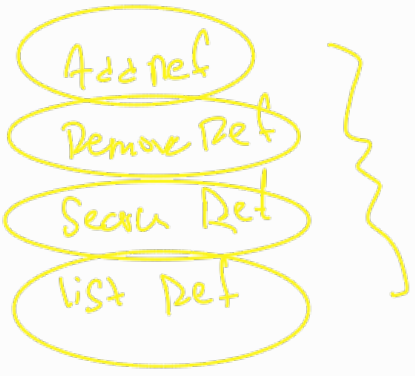
extend → optional

---

generalization → extending 1 usecase into its child usecases



Note:-
Actors are Singular.
Student ✓ (students ✗)



Too fine usecases (Smell)

Right amt usecase

note: A usecase should be a verb.

Don't use Real names.

Practice

usecase → gives a high level view of system

1) Identify the Actors
2) Identify the use cases
3) Identify what the Actor performs with the System

usecase can have only 3 types of connections

1) generalisation
2) Exclude
3) Include

Avoid showing comm b/w actors

1 usecase = 1 user goal

Description template

Identifier — UC-107

name — name of our usecase

purpose — what it is doing

Priority — high/low

Actors — List of Actors Associated with this usecase

Pre_cond

Post. cond

Dependencies — If our usecase lies on (1 incase

Typical course of Action

Actor Action	System Response

Alternate course of Action

Actor Action	System Response

Exceptions → things go wrong

Domain model: → Shows imp real world objects and how they are connected.

Uni – Systems

Objects
→ Student
→ Teacher
→ Department

Relationship
A student enrolls in courses

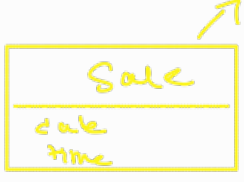
Domain model → foundation for coding

visualizing domain models we use → Analysis Class diagram.

Domain Models only define
↳ Domain objects
↳ Associations b/w them
↳ Attributes
↳ conceptual classes
↓
real world thing.

no methods / no functions

concept & Symbol



"A Sale represents a transaction"
↓
Intension / definition



↓
extension

What this conceptual class applies

Benifits of Domain Model

→ visualy Big pic

→ Ensure Structure and Real world implementation matches

Usecase vs Domain

↓

describes actions

Like going through your car manual what action we can perform / workflow

→ parts of the car (what the System is made of

→ Noun / Description of the System.

Relationships

→ Dependency (weakest Relationship) one class use another Briefly.



order has a dependency on payment

→ Association : Simple connection : Doctor, patient yet they exist Independently.

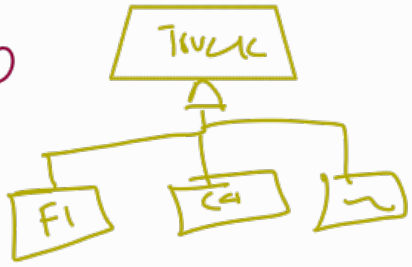


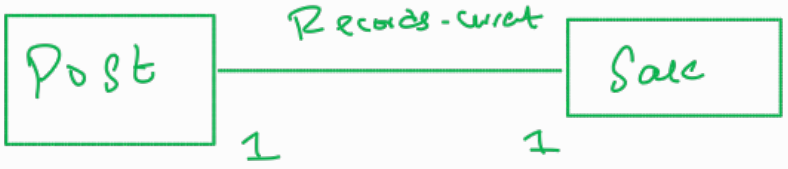
class A can look at class B, But class B doesn't know class A.

→ generalization.. (Inheritence)

Parent ← Child

TRUCK ← Ford F-150







the order is dependent on payment. Order knows payment But payment doesn't know.

1..* → one or more | atleast 1 must exist

* → zero or more | e.g 100 students, 1000....



Exactly 3 or 5 or 8.

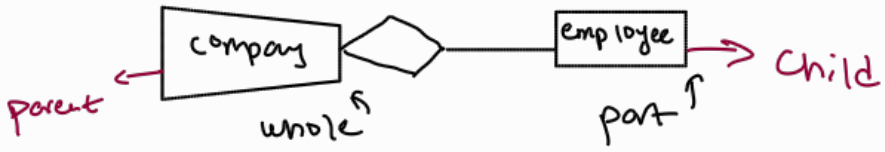
3..5 → min 3 max 5.

Aggregation: one thing is part of another while still independent.

The parent doesn't own the child

↓

class with a class.



Composition ownership is involved. Parents own the childs

Creating a Domain Model

→ find conceptual class ⟶ Identified through nouns.

→ Draw them as classes

→ Add Links | attributes

→ Association name should be Verb-phrase

Give all the nouns

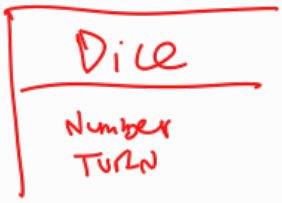
Make sure classes name are realworld so that bussiness can easily know the terms.

Doubt b/w whether it is a attribute (class.)

if doubt whether association is required drop it

↓

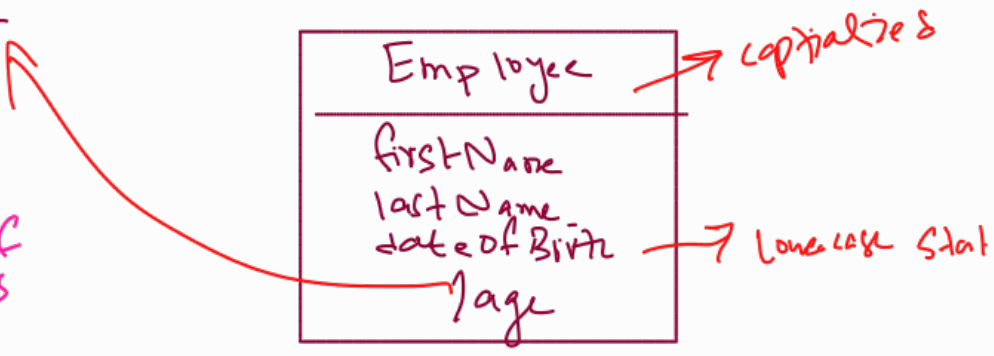
Always make it a class.



Derived Attribute

↓

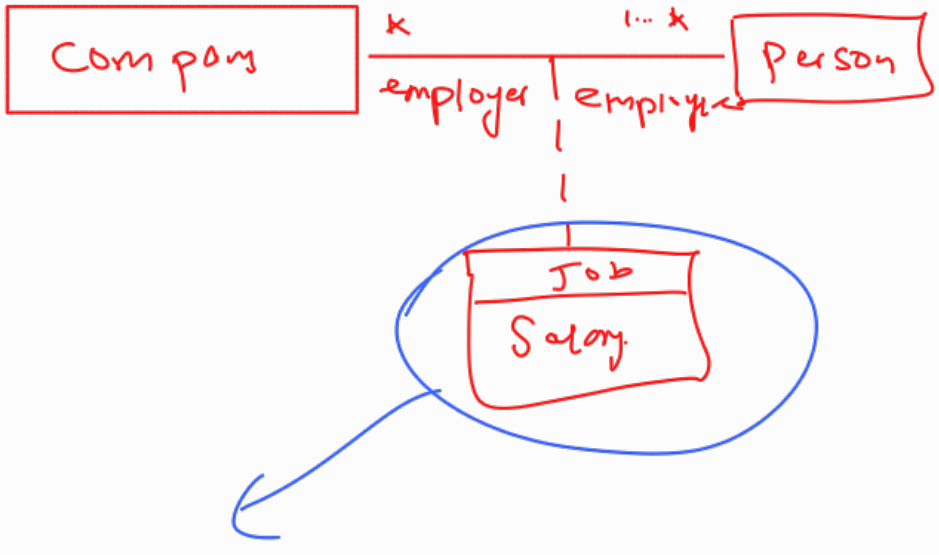
a attribute calculated from other attributes of class



(Association classes) used when more attributes doesn't fit in any 1 class alone.

(use dependency to link together)

dotted line



Association class → Belongs to the Relationship itself

* many - many relationships
* extra data.

To read multiplicity always read the no near the other class.

(Usecase Description template)

→ Identifier — UC-01
→ Name
→ purpose → what is purpose of this usecase.
Priority
Actors
Pre-cond
Post-cond
Dependencies
Exceptions → System failure etc

Typical course of action (primary flow)

Actor Action	System Response
customer clicks checkout	System shows order summary
customer enters payment details	System verifies payment

for 1 usecase only.

e.g here was checkout.

good program: → keep your code short and simple

e.g Each function should do only 1 job specifically (Single Responsibility principle)

↓

Reliability

e.g program should give correct results

Handle invalid input

→ Readability: | code should be easy to read.

```
def f(x,y): (x)
def sum of two var(num1, num2);
```

Abstraction → Hide unnecessary details | user doesn't need to know how a specific feature works

Simple, maintable, Reusable, Scalable, Reliable, Readable, Robust, Abstract, efficient, Secure, Adaptive, Testable

Activity diagram. Shows step by step how tasks work.

Start → Login → Select item → Pay → end

Action nodes → represents tasks being performed. e.g enter pass

Control nodes → controls how the flow moves from one step to another

Shows all sequence of steps to complete a task

Initial/final nodes

Decision/merge
- ↓ Splitting flow based on condition
- → Joining paths again

Fork/Join
- ⇓ Splits flow into parallel tasks
- ⇒ Combines parallel tasks back together.

Objectnodes: Represent data

note:. Some activities run forever, until a external event stops them e.g A security camera system.

completion event:
means when one activity finishes the next step can start.





→ action node represents a specific action that needs to be performed

Control nodes



Start point

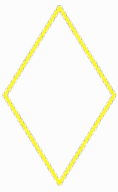


Activity final node

end point (entire activity stops)



flow final node means 1 particular path stops but other paths can continue



decision box → to use this make sure 1 condition is true

has 1 incoming and multiple outgoing flows



which way have guard cond

but it redirects the flow into 1 outgoing / you pick only 1 path

if multiple conditions are true (non-determinism) it causes error.

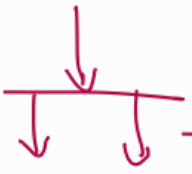
Merge nodes:



no waiting at merge node. if any conditions out of 3 happens it out goes.

concurrency → doing things at the same time

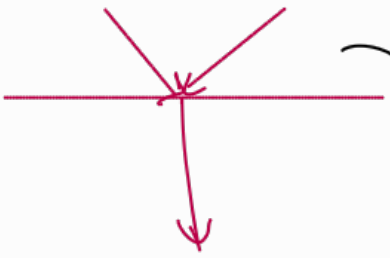
Fork/join used in concurrency meaning when multiple activities happen at the same time



→ Both outgoing paths Run. at the same time

would

Join

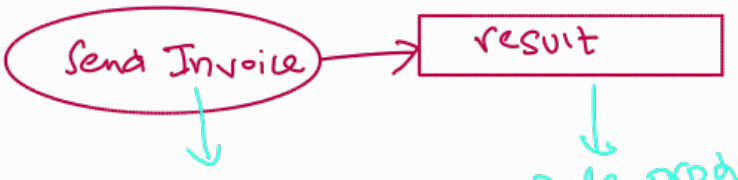


Synchronization needed.

wait for parallel activities to finish then continue

↑ (Both)

(object nodes) represented by a rectangle captures the output of action



action being done

Data produced /output after Sending Invoice.



Send Signals



recieve Signals

← used when dealing with an outside System

Signal → way for one activity to tell another activity to start or continue next activity

after Sending move on the next action node

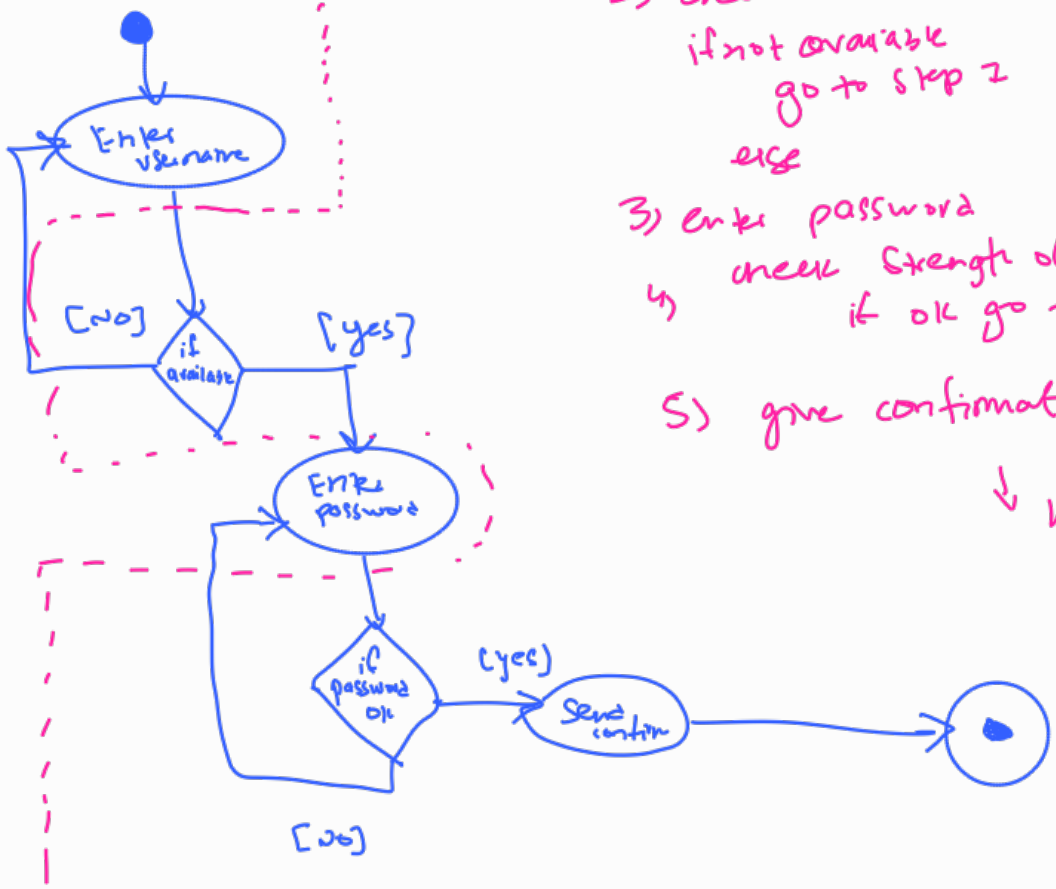
don't move to next action until signal recieved.

note every Arrow leaving  must have a guard condition

usecase → create account

1) Enter username
2) check name availability
   if not available
      go to step 1
   else
3) enter password
4) check strength of password
   if ok go to step 5
5) give confirmation

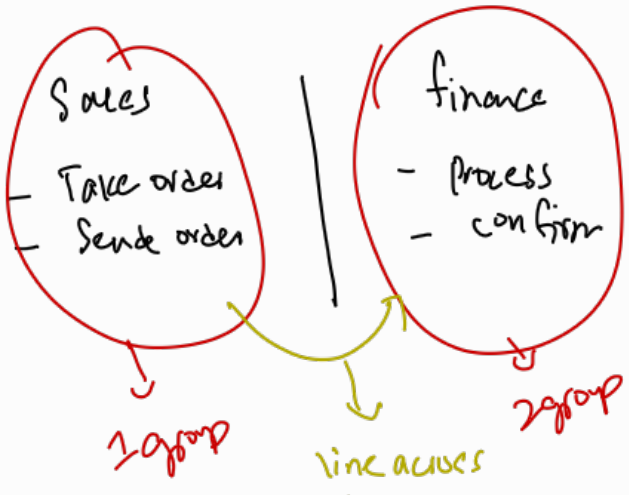
↓ Workflow



Done by user

Done by server

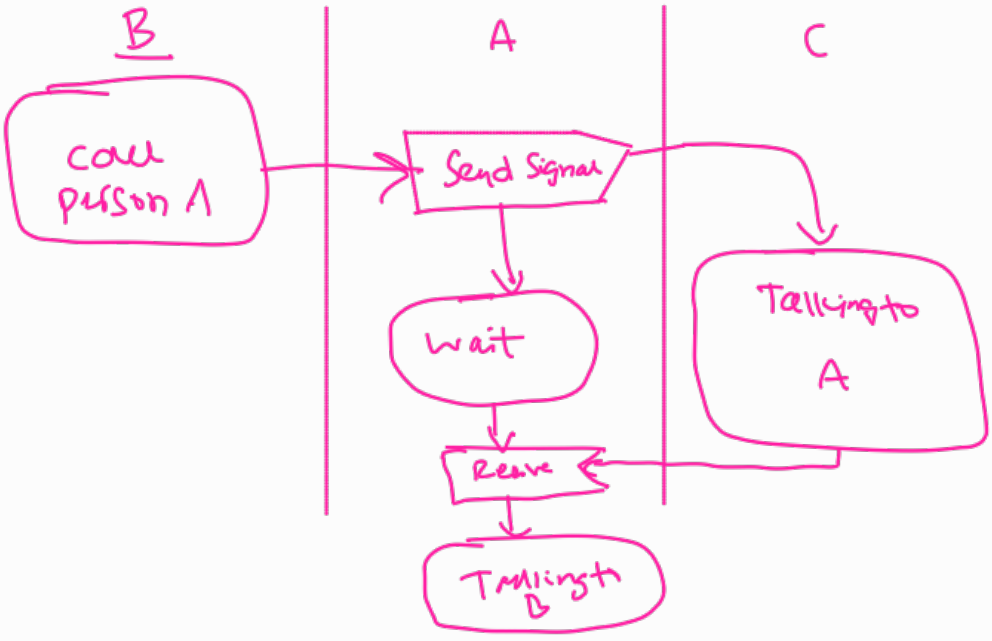
Swimlane → Separate columns in activity diagram. | who each step is doing

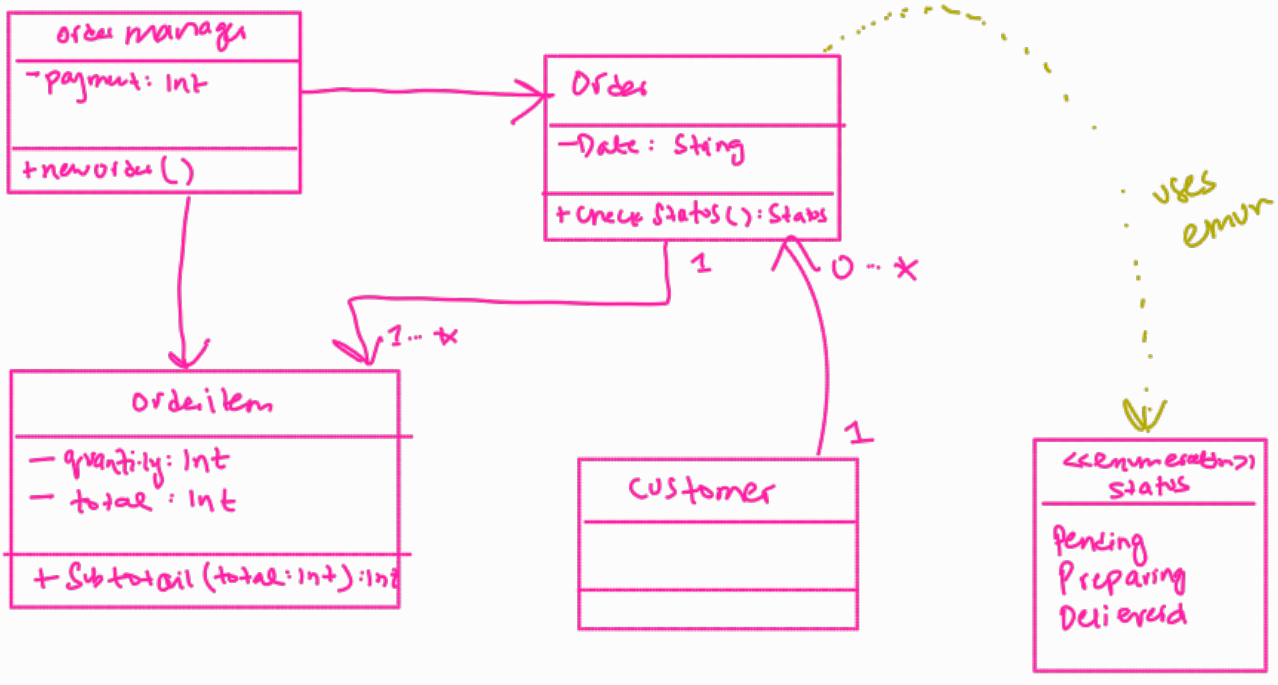


Activity diagram → shows workflow to complete a usecase | Sequentially

↓

and also start from first interaction


B
A
C
call person A
Send Signal
Wait
Talking to A
Reave
Talking to B


order manager
-payment: int
+neworder()
Order
-Date: String
+Check status(): Status
uses enum
1
0..*
1..*
1
orderitem
-quantity: int
-total: int
+Subtotal(total: int): int
customer
<<enumeration>>
status
Pending
Preparing
Delivered

Design class diagram:

Diagrams used for analysis
- → usecases
- → Activity diagram
- → Class diagram (diagrams)
- → System sequence

used for understanding the system.

For Design

→ Interaction diagram | How objects will interact,

Building the System

(Analysis → Design → Implementation in oop)

Design - Class

in Domain Model we only Display real world diagrams related to Business & BA in Design class diagram we will also implement the working in it.

classname
+ Attributes +
methods()

Creating DCD

1) Identify classes that will be used
2) Add attributes / methods

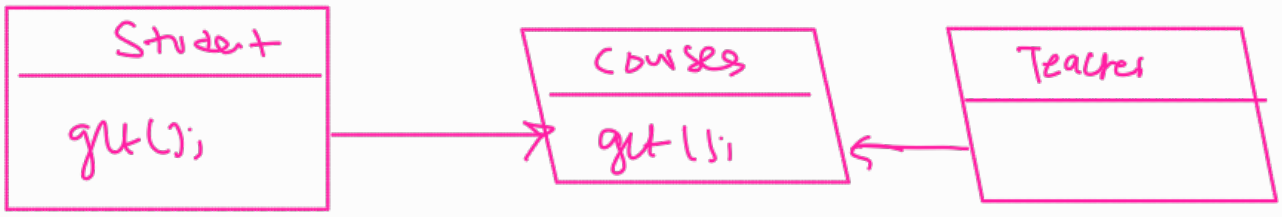
\+ : public
\# Protected
\- : Private
~ Package

Register
- Price - itemID
+endsale() +gettotal()

Register
-Price : Int → DT -Score: Int = 0 → Default value
-method1(P1:String, P2:Int): Foo

3) Add associations

note:: DUD is made from class analysis diagram



Visibility control (from Student we can view courses)

from teacher we can view which courses are being taken by the teacher

note if class A points to class B then it must contain a member variable of class B.

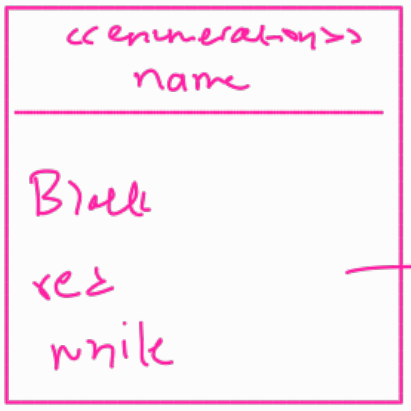
(Which way should Arrows point)

→ Independent object → Dependent

→ 1 ⟶ Many in multiplicity.

enumeration class

↳ Define our own type



, used in status like pending etc

→ values tell only the valid choice in that datatype

Sys Sequence diagram::

Sys Behavior → what does the sys does, without explaining how it does.

SSD

- Shows Interaction b/w Sys ⟷ actors
- Draw using uml
- can also show alternate flows (like wrong pass.)

---

usecase → story to achieve something

Series of interaction b/w actor and the Sys to carry out 1 single usecase

Creating SSD::

→ Drawing a diagram that shows how a user (actor) and System talk to each other.

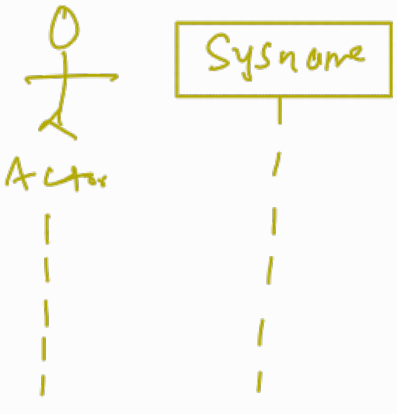
Main Success Scenario → everything works fine

Alternative Scenario → exception/errors.

Black Box (System)
Actor(s)
External Sys
Messages
Sequence (order of events)
T→B

event → action from actor.

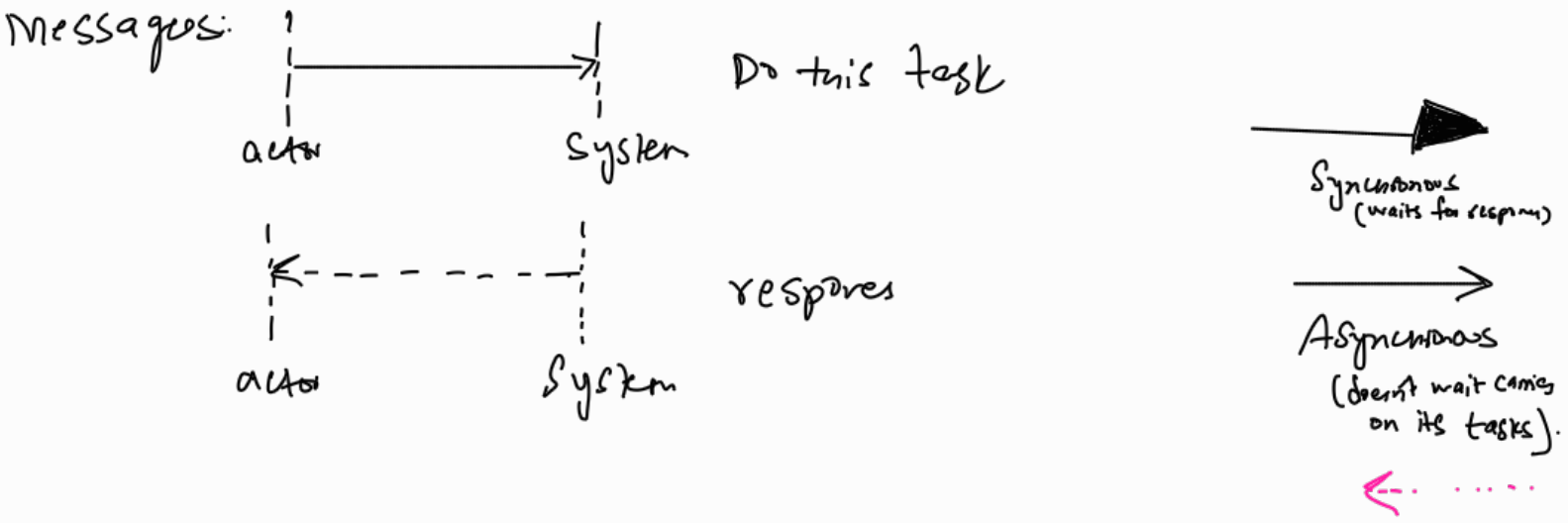
Lifeline → vertical dashed line (who is involved in the interaction)

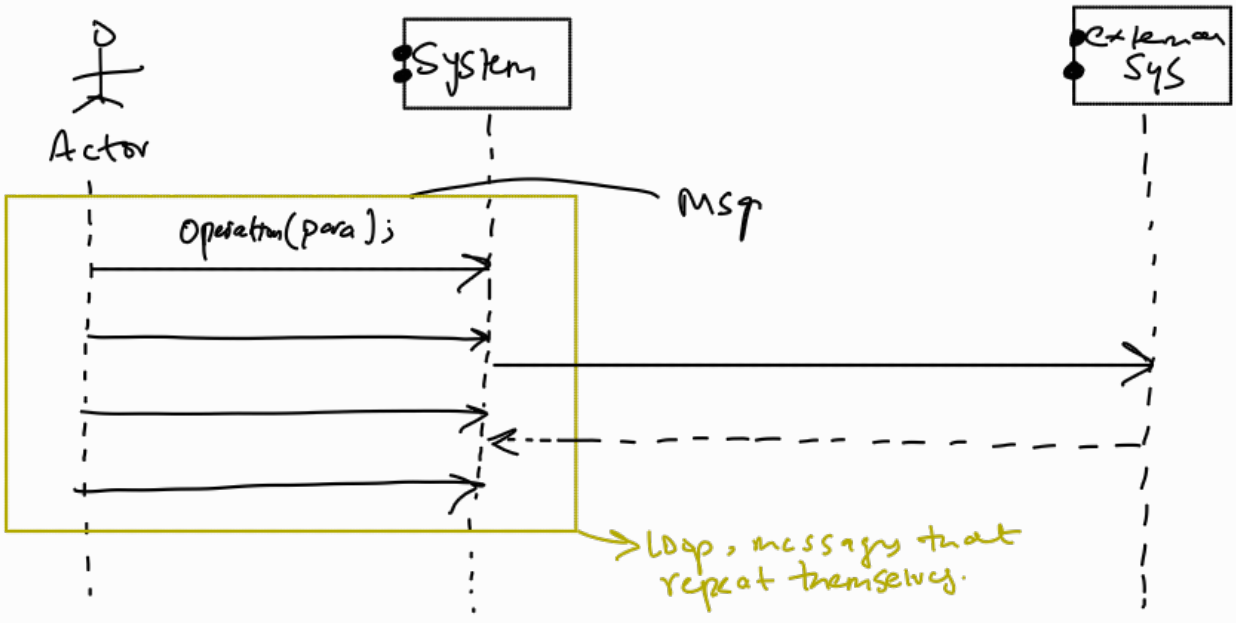




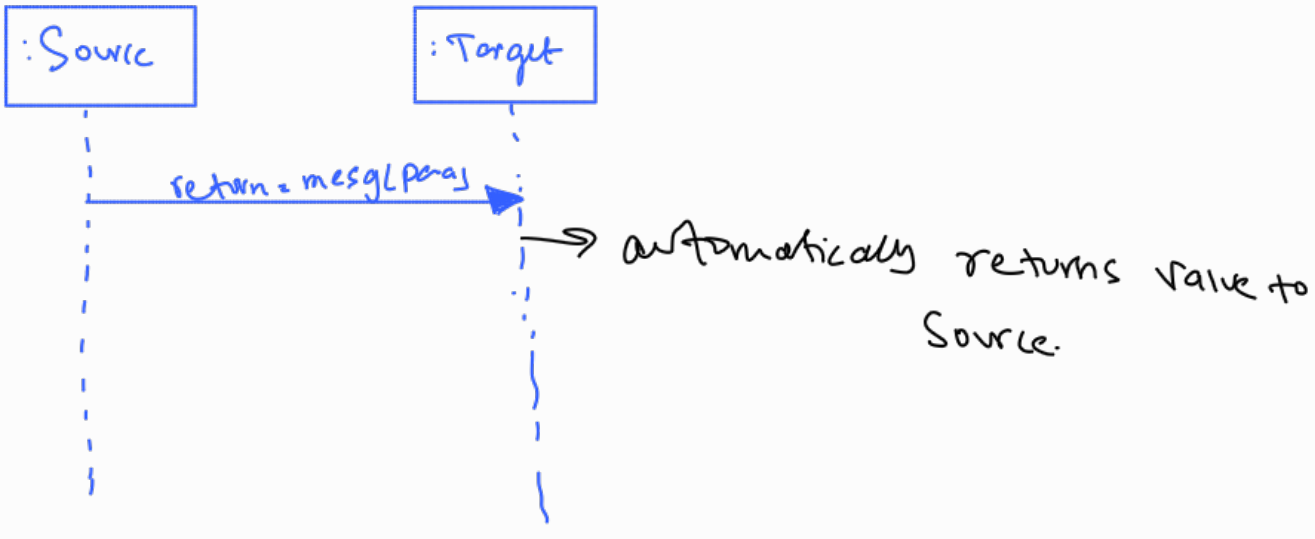
Activation Box

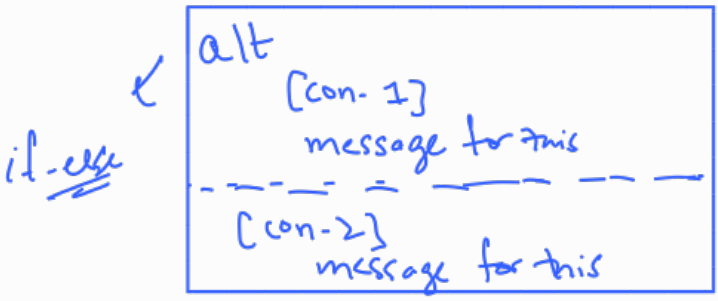
→ a box on the activation box represents period of time the thing is active

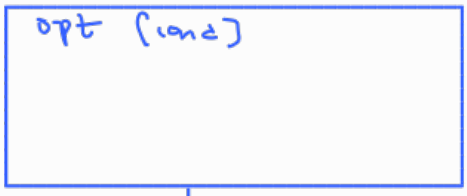




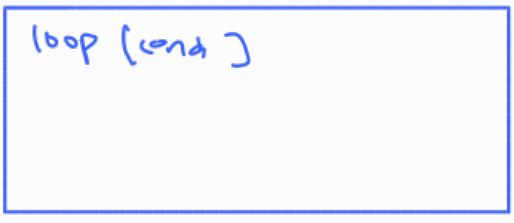
The messages are written in form of method.





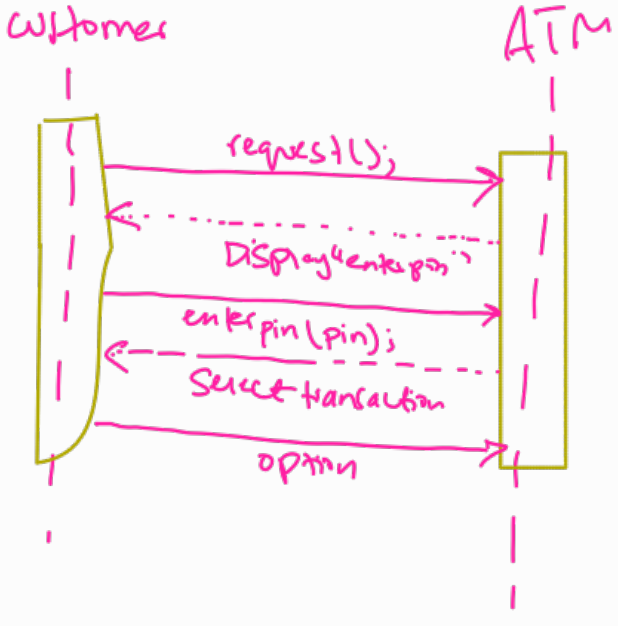


this run if cond is true



Run n times
while cond is true

SSD



Perform transaction

Arrow going in will always be a method

State diagram:- How something changes over time

1) event → Something happens
picking up a phone

An event causes State change

2) State ⇒ condition at a certain time
idle, Active

State A → (Event) → State B

3) Transition → movement.



Types of event
1) normal event
2) error event
Something goes wrong
eg motor Jammed.

3) Related events (dependent)

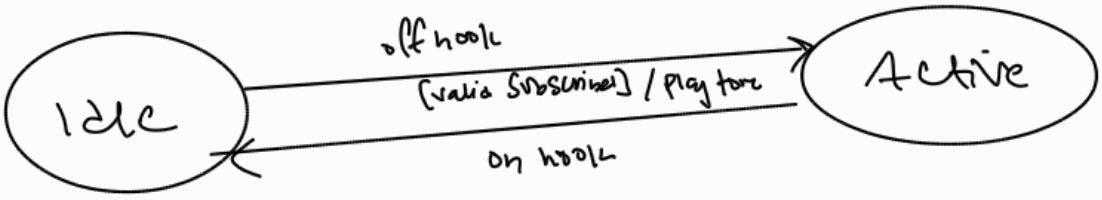


4) unrelated events
no connection

for change in event
the condition to happen → True

When (roomtemp < heating point)

Time event



offhook transition is succeed and if valid Subscriber is true → playtone occurs.
and only then it can move to next state that is Active

Paper Jam
do / flash warning

here if a printer gets stuck in paperjam state, the do will keep on flashing warning as long as a external event doesn't change the state


Payment
pending
state
after 2 days
cancelled
state

Solid principles → guidelines for clean code | focuses on how class interact.

↓

main goals

→ tolerate change

→ easy to understand

→ Reusable code

SRP → 1 class has 1 responsibility

```
Class UserData { Save to dB(); }
```

↓

Benifits

→ easier to change, control, manage and avoid conflicts, and logic Mix naho.

openclosed principle (ocp)

↓

we can add new features in our class without modifying our code

Because modifying will Lead to Bugs

Abstract class ⟶ cannot create objects with it

↓

closed for modification

```
class Shape {
Public:
  virtual void draw() = 0;
};

Shape S;x
```

↓ virtual fun that is written in base class and can be Overridden in derived class

```
class Shape {
    Public
      virtual void draw() { };
    };

class rec : public Shape {
      void draw() override { };
    };
```

→ Abstract class

→ our code here to be implemented.

An Interface allows infinite future extensions with no need to even edit our class again.

Any class that only has a fun() virtual void save() = 0;
is a Interface

LSP Liskov Substitution Principle

→ Childclass should behave like parent class.

if we replace parent with child class it should work fine.

When a child is forced to do something it cannot.

```
class Bird {
public:
  virtual void fly()
  {
    cout << "flying";
  }
};
class Penguin : public Bird {
public:
  void fly() { throw "error"; }
};
```

✗

Parent class says all birds can fly, Penguins say cannot fly
So ✗ LSP

if parent says "this action works"
→ child must make that action properly

Same as OCP, we break the Bad Inheritance classes and give them a common interface

ISP Interface Segregation Principle

A class should not be forced to depend on methods it doesn't use

We design interfaces according to what dif classes need.

```
class IMachine {
    public
        virtual void print() = 0;
        virtual void scan() = 0;
        virtual void fax() = 0;
};
```

(X)

```
class SimplePrinter : public IMachine {
    public
        void print() { // }
        void scan() { // }
        void fax() { // }
};
```

X Simple printer doesn't need

Seperate all virtual functions into there classes and Inherit them to classes that only use them.

Chop this into smaller interfaces and inherit to only classes that use them.

DIP A high level class (general class does knows what will the program do But doesn't care how the logic is implemented) should never be linked to a low-level class directly.

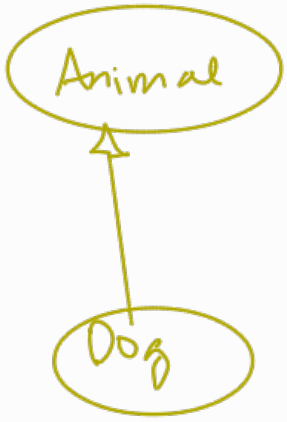
or the things will become tightly coupled; if you change low-level logic it will shatter high level class

Here for that we use a interface class, in DIP always use a pointer to object in the high level class.

In interface write those function that are common in Both low-level classes.

→ interface

```
class Animal {
    virtual void make() = 0
};
class Dog: public Animal {
    void make() override {
        cout << "Boo";
    }
};
```



When overriding a method point child → parent

Metrics → Look if our code is good & Bad. By focusing on

1) cohension (e.g a kitchen class should hold oven and fridge, if you suddenly find carbyre then cohension is low).

High cohension → class does 1 job perfectly.

2) High coupling → dependency b/w classes, meaning if we change class A it breaks class B, C.

3) Size / depth

Object. oriented Metrics?

↓

grades for your code / checks the quality of code

Usage

→ detect Bad design

→ find code mistakes.

Types CK, LK, MOOD → check design quality

CK → check how complicated each class is

LK → Check each class + methods

MOOD → $\frac{\text{total methods + inherited}}{\text{no. of classes}}$

((K tools)

1) WMC → Average weighted methods → more methods ↑ more complexity → High WMC violates SRP

2) DIT How deep a class is in inheritance ↑ depth ↑ more Inherited Stuff. ↓ more complexity

3) NOC → More children, More responsibility | subclasses all pointing to parent class | if tiny change made to parent class, it breaks all children

4) CBO → How many other classes your class depends on.

Class Car { engine; wheels; }

Car class depends on engine, wheels | High coupling makes code less modular

in uml it simply no of connections of each class

5) RFC = methods inside class + methods it calls.

```
class demo {
    void process() {
        m.add();
        m.square();
    }
};
```

= (3) → Bad

↑ RFC → Increases complexity

6) LCOM

Do methods work together | using same data or not?

if methods use diff data ⇒ Bad

LCOM:- P − Q → pair of methods that share atleast 1 var

↓

pairs of methods that share zero var

```
Class report {
    int userage;
    int printerID;

    void method A() { userage = 20; };
    void method B() { if(userage > 17) ; }
    void method C() { printerID = 5; }
}
```

Q → A and B

P → A and C
B and C

LCOM = 2 − 1 = 1 → cohesion exist, |

Lower LCOM → focused classes.

cohesion ↓

does class functionalities go towards a common goal.

Low cohesion + high coupling is Bad.

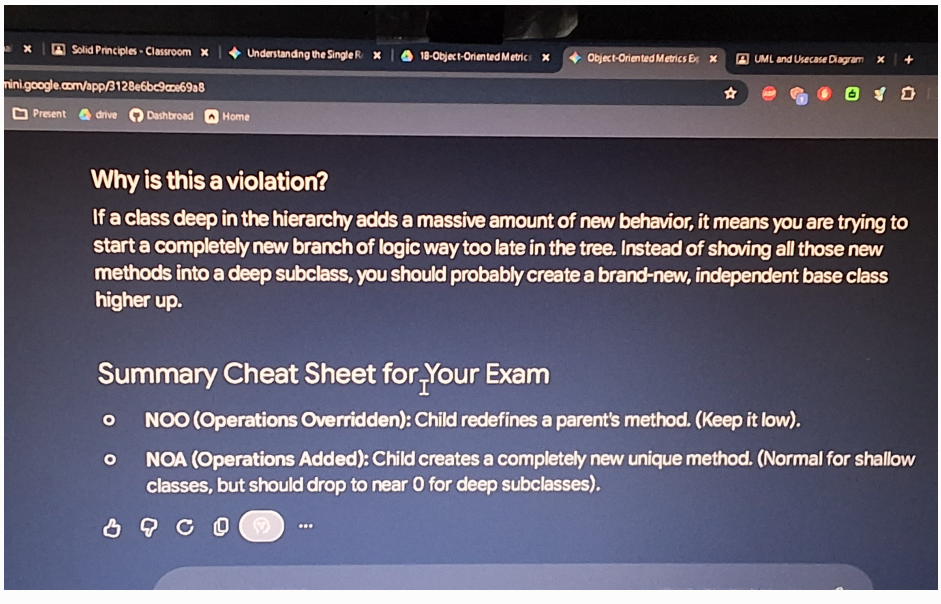
Lx metrics → used in assessing individual code while CK that is used in high level design

CS → classsize : Total no of methods + Local Attributes + Inherited Attributes

NOO → no. of operations overridden by a subclass → larger NOO values indicate Design problems.

NOA → no of operations added by Sub-class

AS DIT increases NOA should decrease first



Specialization Index → tells how mutated our child class has become as it goes deeper into the inheritance tree

$$SI = \frac{NOO \times depth}{\text{Total methods}}$$

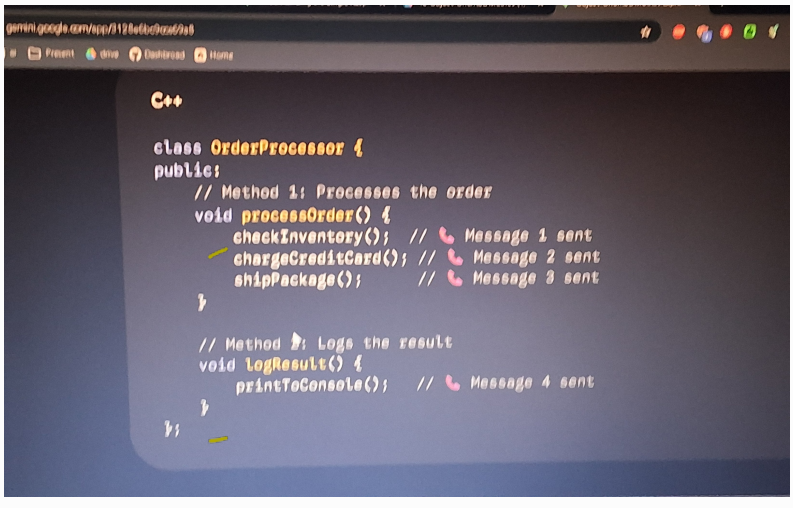
→ Plus marked -ves

OSavg

Avg operation Size

Total External method calls made by class

Total no of methods written inside class



calls → 4

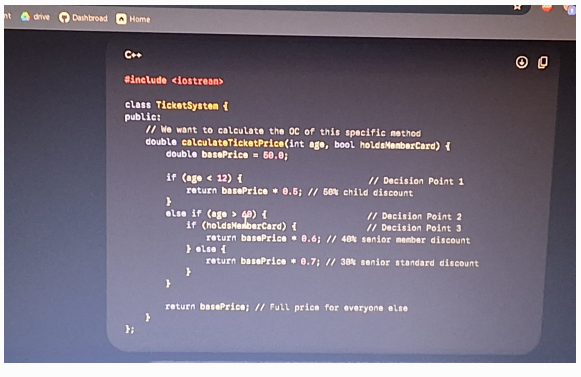
methods 2

$\frac{4}{2}$

OC → operation complexity → checks the diff execution paths in a class method

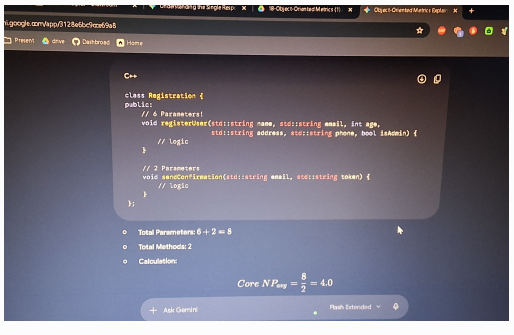
OC = no of decision points +1

if | elseif | for loops | switchcase , ?: ||



3+1 → 4

NPavg → Avg no of parameters → Total no of para across all methods
---
Total no of methods written in class



$\frac{8}{2} = 4.0$

Design Patterns → repeatable Sol to a commonly occuring problem. /
↓ tool/mtd, Rule
gives us a Blueprint / Strategy to Solve a problem.

→ Parts of pattern :-

1) Name :- eg singleton, observer
2) problem :- when to use
3) Solution :- classes / obj you need.
4) consequences :- what did we gain / lose

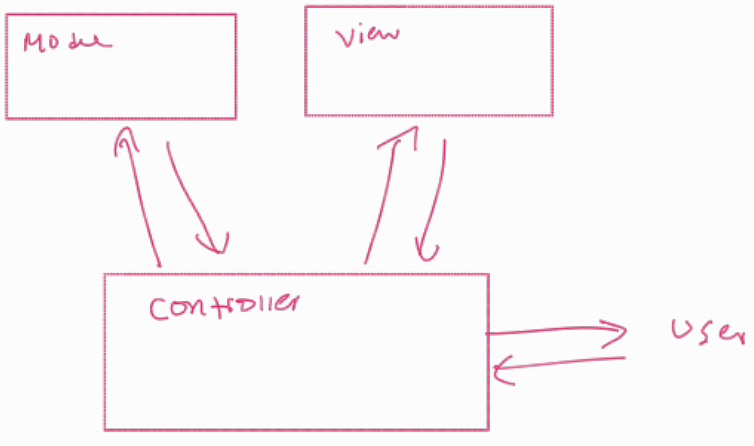
Benifits	DrawBacks
☆ Reusability	complexity,
☆ maintainability	Rigidity,
☆ Abstraction ↓ hiding complex details	Anti patterns ↓ misusing patterns to fine Sol
☆ encapsulation :- Binding of data into single class.	

---

1) MVC → Model View Controller
output / what is displayed / face of our app
→ Bridges the gap b/w view and Model.

↓
Application object / Brain where logic is written
(holds our data)

Decoupled → not glued together. / Model and view are Seperate.



↓
meaning if you
Change the Logic / data
it won't Break the Screen (view)
Instead it will update real.

Design Patterns are grouped into 3 categories

- Creational → Create objects that is easy to Reuse
- Structural → Assemble the objects together
- Behavioral → How objects interact/communicate with each other

Scope → where does the pattern apply to.

1) Class Scope → uses Inheritance. Static.
2) object Scope → Deals with objects | Dynamic



Reuse Mechanism:: → taking your existing code and use it to Build new features.

ways to do this:: | Recycling your existing code

2 ways to do

1) Class Inheritance:: → static → new class is defined Based on a existing one/ Parent
   - Basically extend features of your parent class
2) object composition → Plugging together diff objects.
   - dynamic
   - Better as more flexible/ easier to change
   - Basically class within a class.

(Adapter Pattern) → allows 2 pieces of software that speak diff Lang to work together without changing either one of them.

Advantages:

→ zero disturbance
→ flexibilty
→ Clean code

Singleton → creational design pattern.
→ one country → one president | global

change constructor from public to private

→ Making 1 object from the class, not just one class itself.

Behavioral Design pattern | loose coupling

observer → notified object about any event happened to publisher.

follows OCP

cons: objects are notified at Random order

```
class AnytypeSub {
    Public:
    virtual void Subscribess (const String msg) = 0;
};

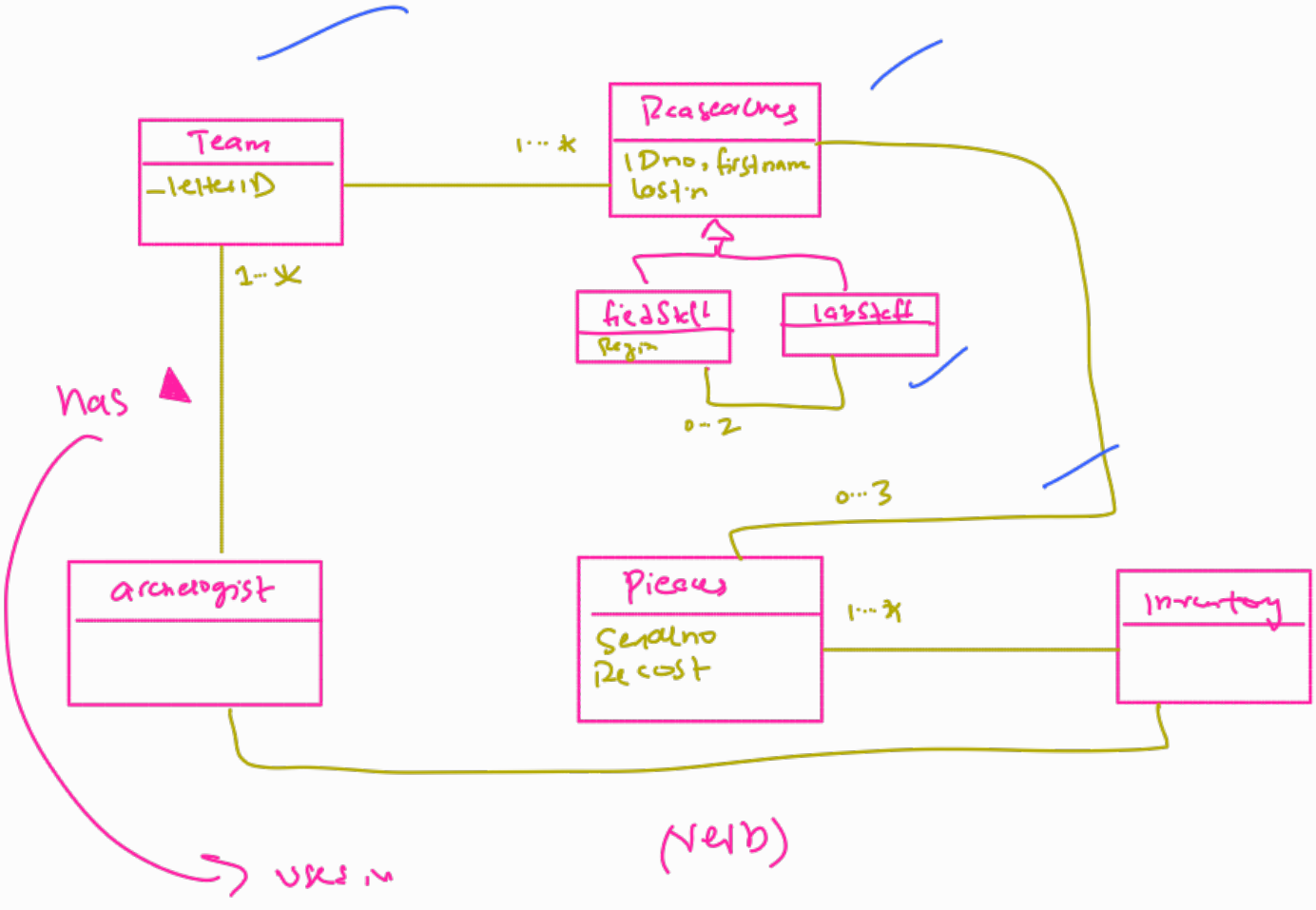
class ActualSubscriber: public AnytypeSub {
    Public:
    Void Subscribers (String msg) override {
        ----
    }
};

class publisher {
    AnytypeSub * anytype [];  //
};
```

our objects

and use names from the real world (Business)

Management System



(NeID)

uses in

Association, Aggregation, composition any because

in dependency, generalization arrow is already there

iterator pattern → an iterator helping objects to traverse a certain tree without changing the implementation of traversal according to change of tree

```
class Iterator {

    virtual void hasNext() = 0;
    virtual void next() = 0;
};
```

→ Abstract Iterator for traversal

```
class Collection {

    virtual void * createIterator() = 0;
};
```

→ Abstract collection of nodes

↓

creates a tracker and links it to data.

In main

```
Iterator * vectorIterator = vectorCollection->createIterator();
```

↓

Same as

```
Iterator * createIterator {
    //
    return new ArrayIterator;
}
```

State → FSM, meaning at any given time our program can change the State from a set of finite State

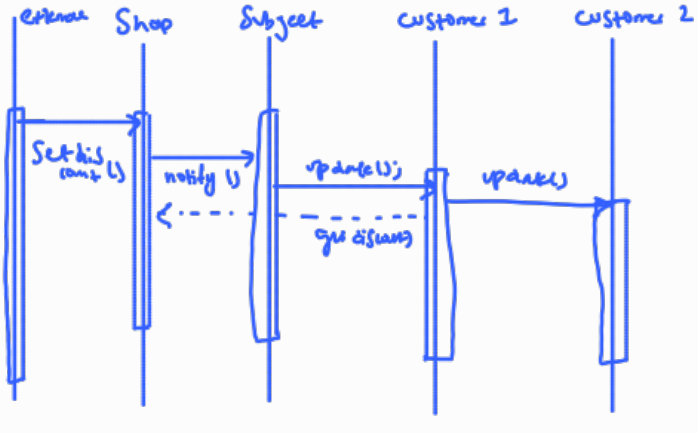
SRP, OCP

Phone → locked → on → unlocked

unlocked → off → locked

For State changes we have to add if-else-if, Switch in program one that is not modular.

Instead turn each State into its Seperate class.



Template pattern